Phase 2: PySR-only against HDS v50_2, and the plan for the Pure LLM baseline

Audit of the `LLM-HypatiaX-REPRO` repository. Branch `audit/step1-cleanup`, Phase 1 tag `phase1-closed` (`9d56d8c`).

> **Status.** Phase 2 is closed *with caveats* for the comparison that was actually run, PySR-only against HDS v50_2. HDS is ahead on recovery, but the primary interval includes zero, so the result is a suggestive direction and not a demonstrated improvement. The comparison Option 3 specifies, Pure LLM against HDS, has **not** been run; Section 6 is its run design.

How to read this report

The Phase 2 rows (`p2/`, `p2_med/`, `p2_train/`), the closure outputs and `scripts/phase2_close.py` are committed on `audit/step1-cleanup` (commit `7c3e6fa`). The far-extrapolation figures in Section 2 were recomputed independently from the 90 `p2/` rows without using the closure script, and agree with its output. The medium and train figures are the closure script's own output, regenerated after the label fix described below. Percentages are per-equation means over three seeds; counts such as 32/45 are rows.

1 What was compared

- **Systems:** PySR-only (A) and full HDS v50_2 (B, the `hypatia` block with the LLM enabled).
- **Cases:** 15 equations $\times$ seeds 42, 99, 123, so 45 rows per system.
- **Scorer:** one wrapper for both (`exp1_ablation._safe_r2`, which calls `r2_scored`) over the frozen `hypatiax/core/r2_canonical.py`. Its SHA-256 (`7d5395ed…`) matches both the run configuration and `triage_out/freeze_manifest.json`. Sign-flip credit is off.
- **Primary field:** the row's `r2` is `extrap_r2_far`, the far-extrapolation score, checked on all 90 rows. Medium extrapolation and training R^2 are sensitivity fields.
- **Split parity:** holds by construction of the code, not by the stored ID. `split_id` in these rows is `derived:` plus a hash of "equation|seed", so it is identical for both systems whatever data they saw. In `exp1_ablation` every condition regenerates its data from the same `(equation, seed)` (`RandomState(seed)` and `train_test_split(random_state=seed)`), so the splits do match. The closure script's identical-`split_id` check is therefore not evidence of this.
- **Unit of analysis:** the equation. Seeds within an equation are replicates, not independent observations.

2 Results

Difference is HDS minus PySR-only, in percentage points. Intervals are paired bootstrap intervals over the 15 equations. The sign test counts only equations where the systems differ (HDS better / PySR-only better / tied).

Sign-test p-values (two-sided, from the closure output): far, $R^2 \geq 0.99$: 0.375; far, $R^2 \geq 0.999999$: 0.219; far, catastrophic: 1.000. For the train recovery row the counts 2/1 give

Field	Metric	PySR-only	HDS	Diff (pts)	95% interval	HDS better / worse / tied
far (primary)	recovery, $R^2 \geq 0.99$	71.1%	82.2%	+11.1	[−2.2, +24.4]	4 / 1 / 10
	recovery, $R^2 \geq 0.999999$	13.3%	35.6%	+22.2	[0.0, +46.7]	5 / 1 / 9
	catastrophic, $R^2 < -10$	15.6%	15.6%	0.0	[−6.7, +6.7]	1 / 1 / 13
medium	recovery, $R^2 \geq 0.99$	71.1%	82.2%	+11.1	[−2.2, +24.4]	4 / 1 / 10
	catastrophic, $R^2 < -10$	8.9%	4.4%	−4.4	[−13.3, 0.0]	1 / 0 / 14
train	recovery, $R^2 \geq 0.99$	91.1%	93.3%	+2.2	[−4.4, +8.9]	2 / 1 / 12
	recovery, $R^2 \geq 0.999999$	0%	0%	0.0	[0.0, 0.0]	0 / 0 / 15
	catastrophic, $R^2 < -10$	0%	0%	0.0	[0.0, 0.0]	0 / 0 / 15

$p = 1.000$ (computed here from the counts, not taken from the output).

Note on the catastrophic rows. The closure script originally counted any positive HDS-minus-PySR difference as "HDS better", including for the lower-is-better catastrophic and NaN rates. That printed medium catastrophic as 0/1/14 although HDS has the lower rate. It is fixed (a direction-aware count, commit on branch `phase2-fix-labels`); the corrected count is 1/0/14. Rates, intervals and sign-test p-values did not change, because the sign test is symmetric.

Reading.

- The primary interval includes zero, and only five equations separate the systems. The strict-threshold row is borderline (lower bound 0.000, $p = 0.219$) and is exploratory: four metrics were examined with no correction.
- Medium extrapolation repeats the far recovery result exactly. It is the same evidence, not independent confirmation, because a formula that is correct scores near 1 in every regime and a wrong one fails in all of them.
- Training R^2 is close to uninformative. The data carry noise of 5% of the target's standard deviation, which caps training R^2 near $1/(1 + 0.05^2) = 0.9975$. The 0.99 threshold sits just under that ceiling, so the strict threshold cannot be met on this field. Row counts passing 0.99 are 41/45 (PySR-only) and 42/45 (HDS) on train, against 32/45 and 37/45 on far: many formulas fit the training range and then fail to extrapolate.
- Arrhenius and Impermanent Loss fail for both systems. The scorer records scores below -100 as $-\infty$, so mean R^2 must not be reported; use counts by status.

3 Time budget

System	Declared cap	Observed
PySR-only	1,400 s (1 attempt × 1,100 s + 300 s)	about 1,110 s in the one record inspected
HDS v50_2	3,600 s (3 attempts × 1,100 s + 300 s)	median 128 s, maximum 729 s; 0 of 45 rows above 1,400 s

The rows carry two different `timeout_s` values, so the closure script's single-timeout check fails; the run was closed with `--declare-unequal-timeout`. The accurate statement is: *declared budgets differ (3,600 s against 1,400 s), but no HDS row exceeded 1,400 s, so the larger cap did not bind.* This is an observation, not an equal-budget run. It assumes `runtime_s` covers the whole case including retries. In the one record inspected `runtime_s` equals `sr_time_s` while `llm_time_s` (7.3 s) is listed separately, so whether LLM time is inside

`runtime_s` still needs confirming. The median runtime ratio (HDS/PySR-only) is 0.12.

4 What this supports, and what it does not

> **Supported:** on the far-extrapolation field HDS recovered more equations than PySR-only in the sample (+11.1 points), in a fraction of the time, and the direction is the same on the strict threshold.
> **Not supported:** "HDS beats PySR-only" (interval includes zero), "HDS beats a Pure LLM" (not run), any equal-budget claim (declared caps differ), and any claim about the leave-one-gate-out ablations (no result files exist).

Other limits: 15 equations, most of them textbook formulas, so recovery may partly measure recall; the HDS rows came from the `exp1_ablation` experiment and were flattened into `p2/`, not launched as a dedicated Phase 2 run; and Phase 3 cannot reuse this data.

5 Phase 1 preconditions

Phase 2 assumes four Phase 1 conditions. Against the audit in `AUDIT/phase1_audit.json`: generation paths are logged, the hardcoded bypass is closed at source and every probed task shows no bypass, the quarantine manifest exists (64 contaminated result files removed, stored M4 rows without a formula flagged and excluded), and the evaluator is on `r2_canonical` with regression tests. One item stays open by design: the row "Evaluator guard and R2 policy" reads *Unverified* because the comparison of Supplement A Listing 8 with `r2_checked` is a manual step the audit cannot perform. Confirm that comparison before the text claims the evaluator is settled.

6 Pure LLM baseline: run design

This is the comparison Option 3 specifies for Phase 2. **It has not been run**, and the Pure LLM runner was not available when the design was written, so the configuration keys describe the interface the runner needs. The companion files are `phase2_pure_llm_config.yaml`, `phase2_pure_llm_run_spec.md` and `phase2_row_schema.json`.

Decisions to settle before the first run

In the configuration file `temperature`, `max_tokens`, `max_calls_per_case`, `prompt_inputs` and the prompt hash are `null`; the run is meant to refuse to start while any is unset. "Matched" means a matched *cap*, not matched spend: actual spend is reported per row (`runtime_s`, `n_llm_calls`, tokens).

Analysis declared in advance

Primary comparison HDS minus Pure LLM; primary metric recovery at far $R^2 \geq 0.99$, mean over seeds per equation; interval and sign test as in Section 2. A directional claim is written

ID	Decision	Recommended default
D1	Time budget B	1,400 s wall-clock per case for all three systems
D2	How HDS meets B	R0: reuse the 45 rows as compliant by observation. R1: rerun HDS with `timeout_s` = 1400, only if the text must say "equal budget"
D3	Pure LLM prompt inputs	Exactly what the HDS LLM stage receives. **Must be decided**; if either prompt contains the equation name, recovery partly measures recall
D4	Constant fitting	None; score the formula as emitted. A fitted variant is a separate labelled arm
D5	Sampling	One sample per (equation, seed); temperature and `max_tokens` equal to the HDS LLM stage. **Must be decided.** The API may have no sampling seed (verify), so `seed` fixes data only
D6	Retries	Fixed maximum, all inside B and counted in `n_llm_calls`. **Maximum must be decided**
D7	Timeouts	Row kept with status `timeout`, `r2` null, counted as not recovered

only if the primary interval excludes zero. With five non-tied equations the smallest two-sided sign-test p is 0.0625, and six non-tied equations all favouring one system give 0.031, so power comes from the number of equations, not seeds. Strict threshold, catastrophic rate and the sensitivity fields are exploratory and are not promoted if they look better. Changes after the first row is written go in `phase2_pure_llm_amendments.md` with a date.

Gates

Phase 2 for this comparison closes only if all of these hold:

Gate	Condition
G1	the data split equals the HDS one for every (equation, seed), compared by hashing the data arrays, not the derived `split_id`; no case without an HDS counterpart
G2	`r2` equals `extrap_r2_far` in every row
G3	`llm_called` true and `bypass_used` false in every row
G4	one `timeout_s` (equal to B) and one `prompt_sha256` across all Pure LLM rows
G5	`formula` stored for every row with status `ok`
G6	exactly 45 rows, no duplicate (equation, seed)
G7	`model_id` and `evaluator_version` identical to the HDS rows
G8	`code_dirty` false and a single `git_commit` across rows
G9	every row has `r2_raw` and `r2_status`
G10	`phase2_close.py` runs with no FAIL other than a declared one

Cost cannot be estimated until a dry run records calls, tokens and runtime per case; multiply by 45 for the full run. Under R1 add the HDS rerun: about 1.6 hours at the observed median runtime, up to 9.1 hours if every case used the full 1,400 s.

7 Verification status and reproduction

Item	Status
`r2_canonical.py` SHA-256 equals the configuration value and the freeze manifest entry	checked
Tag `phase1-closed` points at `9d56d8c`	checked
`reflatten_fields.py`: verifies, writes `p2_med`/`p2_train`, stops with exit 2 on a planted mismatch (synthetic fixture, not real rows)	checked
Far-extrapolation rates and counts in Section 2, HDS runtime (median 128 s, maximum 729 s, 0 of 45 above 1,400 s)	recomputed independently from `p2/` rows
Medium and train rows in Section 2	closure script output
Catastrophic “better/worse” labels	fixed, regenerated
Supplement A Listing 8 against `r2_checked`	manual, open
Whether `runtime_s` includes LLM time	to confirm

Commands (from the repository root):

```
# verify that r2 is extrap_r2_far on every row (writes nothing)
python scripts/reflatten_fields.py --check-only

# build the sensitivity sets
python scripts/reflatten_fields.py --sens p2_med=extrap_r2_medium \
    --sens p2_train=train_r2 --force
# primary closure; for a sensitivity set use p2_med/ or p2_train/ and a separate --out-dir
python scripts/phase2_close.py --pure-llm p2/pysr_*.jsonl --hds p2/hds_*.jsonl \
    --a-name pysr_only --out-dir phase2_out \
    --declare-unequal-timeout "HDS: 3 x 1100 s + 300 s = 3600 s; PySR-only: 1400 s"
```

Exit codes of `reflatten_fields.py`: 0 ok, 1 usage or I/O error, 2 primary-field mismatch, 3 source lookup failed. Commit it with `p2_med/` and `p2_train/` so the derived files can be regenerated.

8 Open items

1. Merge the label fix and the regenerated closure outputs (branch `phase2-fix-labels`). Remove the duplicate `scripts/phase2_close_v2.py` (identical to `phase2_close.py`) and the three superseded `flatten_results*.py` files, or keep them deliberately.
2. Confirm `runtime_s` covers LLM time and retries, then restate the budget caveat if it does not.
3. Compare Supplement A Listing 8 with `r2_checked`.
4. Settle D5 (the HDS stage requests 3 candidates per call; decide how a one-formula Pure LLM arm is matched) and D6 (retry maximum), check that `PureLLMBaseline` builds the same prompt as the HDS stage (not checked), freeze the prompt and record its hash, then run the dry run and the full Pure LLM run.
5. Phase 3 (leave-one-gate-out ablations) needs new runs that log `r2_raw`, `r2_status` and `config_diff` per row.